

# Redis — Guide d'apprentissage demi-journée

NoSQL · In-memory · Clé-valeur · Pub/Sub · TTL

## 1. Qu'est-ce que Redis ?

Redis (Remote Dictionary Server) est une base de données NoSQL in-memory. Toutes les données sont stockées en RAM, ce qui le rend extrêmement rapide — latence inférieure à la milliseconde.

**Positionnement** : Redis n'est pas un remplacement de PostgreSQL. C'est un outil complémentaire : cache, sessions, temps réel, files de messages, classements.

### Caractéristiques principales :

- **In-memory** : données en RAM (optionnellement persistées sur disque)
- **Clé-valeur** : accès direct par clé,  $O(1)$  dans la plupart des cas
- **Types riches** : String, List, Set, Hash, Sorted Set, Stream...
- **TTL natif** : expiration automatique des clés
- **Pub/Sub** : système de messagerie temps réel intégré
- **Single-threaded** : pas de race condition sur les opérations atomiques

⚡ Redis peut traiter plus de 100 000 opérations par seconde sur un seul nœud. La latence typique est de 0,1 à 1 ms.

## 2. Types de données

Redis propose 5 types fondamentaux. Choisir le bon type est la compétence clé.

| Type       | Commande clé    | Usage                         | Exemple                               |
|------------|-----------------|-------------------------------|---------------------------------------|
| String     | SET / GET       | Valeur simple, compteur, JSON | <code>SET user:1 "Alice"</code>       |
| List       | LPUSH / LRANGE  | File, historique, log         | <code>LPUSH messages "{...}"</code>   |
| Set        | SADD / SMEMBERS | Tags, membres uniques         | <code>SADD rooms "general"</code>     |
| Hash       | HSET / HGET     | Objet structuré               | <code>HSET user:1 name "Alice"</code> |
| Sorted Set | ZADD / ZRANGE   | Classement, score             | <code>ZADD scores 100 "Alice"</code>  |

### String — le type de base

Stocke n'importe quelle valeur binaire jusqu'à 512 MB. Utilisé pour les compteurs, les sessions, les flags.

```
SET compteur 0
INCR compteur      # → 1 (atomique)
INCRBY compteur 5   # → 6
```

```
SET session:abc "user:42" EX 3600 # expire dans 1h
GET session:abc # → "user:42"
```

## List — liste ordonnée

Doubly-linked list. Insertion  $O(1)$  en tête/queue. Accès par index  $O(n)$ . Idéale pour les historiques et files d'attente.

```
LPUSH messages "msg3" # insère en tête → [msg3]
LPUSH messages "msg2" # → [msg2, msg3]
RPUSH messages "msg4" # insère en queue → [msg2, msg3, msg4]
LRANGE messages 0 -1 # tous les éléments
LRANGE messages 0 9 # 10 premiers
LTRIM messages 0 199 # garde max 200 éléments
```

## Set — ensemble sans doublons

Ensemble non ordonné. Ajout/recherche/suppression  $O(1)$ . Opérations ensemblistes entre plusieurs Sets.

```
SADD tags "redis" "nosql" "cache"
SISMEMBER tags "redis" # → 1 (existe)
SMEMBERS tags # → {"redis", "nosql", "cache"}
SCARD tags # → 3 (nombre d'éléments)
SUNION tags1 tags2 # union de deux Sets
```

## Hash — objet structuré

Map de champs/valeurs. Idéal pour stocker un objet sans sérialiser en JSON.

```
HSET user:42 name "Alice" age "30" role "admin"
HGET user:42 name # → "Alice"
HMGET user:42 name role # → ["Alice", "admin"]
HGETALL user:42 # → tous les champs
HINCRBY user:42 age 1 # incrémente un champ numérique
```

## Sorted Set — classement avec score

Comme un Set mais chaque membre a un score numérique. Trier, paginer, faire des classements.

```
ZADD leaderboard 4500 "Alice"
ZADD leaderboard 3200 "Bob"
ZADD leaderboard 5100 "Charlie"
ZRANGE leaderboard 0 -1 WITHSCORES # ordre croissant
```

```
ZREVRANGE leaderboard 0 2 WITHSCORES # top 3
ZSCORE leaderboard "Alice"           # → 4500
```

### 3. Commandes essentielles

#### Gestion des clés

| Commande                    | Description                     | Exemple                              |
|-----------------------------|---------------------------------|--------------------------------------|
| <code>EXISTS key</code>     | Vérifie si une clé existe       | <code>EXISTS user:42</code>          |
| <code>DEL key</code>        | Supprime une clé                | <code>DEL session:abc</code>         |
| <code>EXPIRE key sec</code> | TTL en secondes                 | <code>EXPIRE session:abc 3600</code> |
| <code>TTL key</code>        | Temps restant (sec)             | <code>TTL session:abc</code>         |
| <code>KEYS pattern</code>   | Liste les clés (danger en prod) | <code>KEYS user:*</code>             |
| <code>SCAN cursor</code>    | Itère sans bloquer              | <code>SCAN 0 MATCH user:*</code>     |
| <code>TYPE key</code>       | Type de la valeur               | <code>TYPE messages</code>           |
| <code>RENAME key new</code> | Renomme une clé                 | <code>RENAME old:key new:key</code>  |

⚠ Ne jamais utiliser `KEYS *` en production — bloque Redis le temps du scan. Utiliser `SCAN` à la place.

#### Commandes atomiques utiles

| Commande                    | Description                  | Exemple                            |
|-----------------------------|------------------------------|------------------------------------|
| <code>SETNX key val</code>  | SET if Not eXists (verrou)   | <code>SETNX lock:resource 1</code> |
| <code>GETSET key val</code> | Lit et remplace atomiquement | <code>GETSET compteur 0</code>     |
| <code>INCR / DECR</code>    | Incrément/décrément atomique | <code>INCR connected:room</code>   |
| <code>MULTI / EXEC</code>   | Transaction (tout ou rien)   | <code>MULTI ... EXEC</code>        |

### 4. TTL et expiration

L'expiration automatique est l'une des fonctionnalités les plus puissantes de Redis.

```
# Définir une clé avec TTL dès la création
SET session:abc "user:42" EX 3600      # expire dans 3600s
SET session:abc "user:42" PX 3600000   # expire dans 3600000ms

# Ajouter un TTL après coup
EXPIRE session:abc 3600
PEXPIRE session:abc 3600000            # version millisecondes
```

```
# Consulter le TTL
TTL session:abc      # → secondes restantes (-1 = pas de TTL, -2 = n'existe pas)
PTTL session:abc     # → millisecondes restantes

# Supprimer le TTL (rendre la clé persistante)
PERSIST session:abc
```

**Cas d'usage TTL :** sessions utilisateur · tokens temporaires · cache HTTP · messages éphémères · codes de vérification · rate limiting

## 5. Pipeline

Sans pipeline, chaque commande = 1 aller-réseau. Avec pipeline, N commandes = 1 seul aller-réseau.

```
// Sans pipeline : 4 allers-réseau
await redis.lpush(key, payload)
await redis.ltrim(key, 0, 199)
await redis.expire(key, 86400)
await redis.publish(`chat:${room}`, payload)

// Avec pipeline : 1 seul aller-réseau
const pipe = redis.pipeline()
pipe.lpush(key, payload)
pipe.ltrim(key, 0, 199)
pipe.expire(key, 86400)
pipe.publish(`chat:${room}`, payload)
await pipe.exec()
```

♥ Le pipeline réduit la latence réseau de façon drastique. Pour 4 commandes à 1ms chacune :  
4ms sans pipeline → ~1ms avec pipeline.

## 6. Commandes atomiques

Une commande atomique s'exécute en une seule étape indivisible — elle ne peut pas être interrompue par une autre opération.

Le problème sans atomicité

```
Client A lit compteur → 5
Client B lit compteur → 5
Client A écrit 5+1    → 6
Client B écrit 5+1    → 6 ← résultat attendu : 7, on a perdu un
incrément
```

## Avec INCR (atomique)

```
INCR compteur # lit + modifie + écrit en une seule opération
ininterruptible
```

Redis étant single-threaded, il est impossible qu'une autre commande s'intercale entre la lecture et l'écriture.

## Pipeline ≠ atomique

Un pipeline envoie plusieurs commandes en un aller réseau, mais elles restent séparées. Pour une vraie atomicité sur plusieurs commandes, utiliser **MULTI** / **EXEC** :

```
MULTI
INCR compteur
EXPIRE compteur 60
EXEC # les deux s'exécutent d'un bloc, sans interruption possible
```

---

## 7. Pub/Sub

Redis intègre un système de messagerie publish/subscribe. Les messages sont éphémères : si personne n'écoute, le message est perdu.

```
# Terminal 1 – Abonné
SUBSCRIBE chat:general
# → (waiting for messages...)

# Terminal 2 – Éditeur
PUBLISH chat:general '{"pseudo":"Alice","content":"Salut"}'
# → Terminal 1 reçoit le message immédiatement

# Plusieurs channels en même temps
SUBSCRIBE chat:general chat:dev notifs:user:42

# Pattern matching
PSUBSCRIBE chat:* # écoute tous les channels chat:...
```

⚠ Un client Redis en mode SUBSCRIBE ne peut plus exécuter d'autres commandes. Toujours créer une instance Redis dédiée pour le Pub/Sub.

## Pub/Sub vs Streams

- **Pub/Sub** : éphémère, temps réel, simple — parfait pour notifications et SSE
- **Streams** (**XADD**/**XREAD**) : persistant, rejouer l'historique, groupes de consommateurs — pour des cas plus complexes

## 8. Patterns courants

### Cache (Cache-Aside)

```
async function getUser(id) {
  const cached = await redis.get(`user:${id}`)
  if (cached) return JSON.parse(cached)

  const user = await db.findUser(id) // lecture BDD
  await redis.set(`user:${id}`, JSON.stringify(user), 'EX', 300)
  return user
}
```

### Rate Limiting

```
async function rateLimit(ip) {
  const key = `rl:${ip}:${Math.floor(Date.now() / 60000)}`
  const count = await redis.incr(key)
  if (count === 1) await redis.expire(key, 60) // TTL 1 min
  return count <= 100 // max 100 req/min
}
```

### Verrou distribué (simple)

```
# Acquérir le verrou
SET lock:resource "token_unique" NX EX 30
# NX = seulement si la clé n'existe pas
# EX 30 = expire dans 30s (évite les deadlocks)

# Libérer le verrou
GET lock:resource # vérifier que c'est notre token
DEL lock:resource
```

### Leaderboard temps réel

```
ZADD scores 4500 "Alice"
ZADD scores 3200 "Bob"

# Top 10
ZREVRANGE scores 0 9 WITHSCORES

# Rang d'un joueur (0-indexé)
ZREVRANK scores "Alice" # → 0 (1er)
```

```
# Mise à jour du score
ZINCRBY scores 300 "Bob" # +300 points
```

## 9. Persistance

Redis est in-memory mais peut persister sur disque via deux mécanismes.

| Mode              | Description                    | Commande                                |
|-------------------|--------------------------------|-----------------------------------------|
| RDB (snapshot)    | Photo complète à intervalles   | <code>SAVE</code> / <code>BGSAVE</code> |
| AOF (append-only) | Journalise chaque commande     | <code>CONFIG SET appendonly yes</code>  |
| Sans persistance  | Données perdues au redémarrage | <code>--save "" --appendonly no</code>  |

Pour un projet de cours, désactiver la persistance pour simplifier :

```
docker run redis:7-alpine redis-server --save "" --appendonly no
```

## 10. Redis avec Node.js (ioredis)

```
import Redis from 'ioredis'

// Connexion
const redis = new Redis({ host: 'localhost', port: 6379 })

// Opérations de base
await redis.set('key', 'value')
const val = await redis.get('key')

// Pipeline
const pipe = redis.pipeline()
pipe.set('a', 1)
pipe.expire('a', 60)
const results = await pipe.exec()

// Pub/Sub - instance DÉDIÉE obligatoire
const sub = new Redis({ host: 'localhost', port: 6379 })
await sub.subscribe('chat:general')
sub.on('message', (channel, message) => {
  console.log(channel, message)
})
```

## 11. Cheatsheet rapide

| Commande       | Description                 | Exemple                |
|----------------|-----------------------------|------------------------|
| SET k v EX s   | Stocker avec TTL            | SET token abc EX 3600  |
| GET k          | Lire une valeur             | GET token              |
| INCR k         | Incrémenter                 | INCR visits            |
| LPUSH k v      | Ajouter en tête de liste    | LPUSH history "page1"  |
| LRANGE k 0 -1  | Lire toute la liste         | LRANGE history 0 -1    |
| SADD k v       | Ajouter dans un Set         | SADD online "Alice"    |
| SMEMBERS k     | Lire un Set                 | SMEMBERS online        |
| HSET k f v     | Stocker un champ Hash       | HSET user:1 name Alice |
| HGETALL k      | Lire tout un Hash           | HGETALL user:1         |
| ZADD k score v | Ajouter au Sorted Set       | ZADD board 100 Alice   |
| PUBLISH ch msg | Publier un message          | PUBLISH chat hello     |
| SUBSCRIBE ch   | S'abonner à un canal        | SUBSCRIBE chat         |
| TTL k          | Voir le TTL restant         | TTL session:abc        |
| DEL k          | Supprimer une clé           | DEL cache:page1        |
| FLUSHDB        | Vider la base (attention !) | FLUSHDB                |

**À retenir absolument :** KEYS \* en prod = danger · Un client par SUBSCRIBE · Pipeline = performance · TTL = données éphémères · Choisir le bon type de données